



Universidad Nacional de La Matanza
Departamento de Ingeniería e Investigaciones Tecnológicas

Programación Concurrente

Trabajo Práctico N° 1: “Métricas”

Fecha de entrega: 08/10/2025

Grupo N° 4:

Apellido y Nombre	DNI
Diaz, Juan Jose	38958153
Dato Firpo, Micaela	39830964
Correa, Juan Pablo	40653000
Federico, Agustín	41882938
Burnowicz, Alejo	42646860

Métricas

*Micaela Dato Firpo, Juan Jose Diaz,
mdatofirpo@alumno.unlam.edu.ar; juanjdz@alumno.unlam.edu.ar
Agustín Federico, Alejo Burnowicz
agusfede99@hotmail.com, alejoburnowicz@yahoo.com.ar
Juan Pablo Correa
correacod@gmail.com*

Resumen

El estudio evalúa el desempeño de modelos de inteligencia artificial generativa en el desarrollo de software, específicamente en la multiplicación concurrente de matrices. Se analizaron dos herramientas: GPT-5.0 y Claude Sonnet 4 en GitHub Copilot, todas aplicadas a un mismo prompt técnico en Python. Los resultados muestran que GPT-5.0 logró un mejor rendimiento, siendo hasta un 24 % más rápido en Python puro, mientras que Claude Sonnet 4 destacó por su estructura clara y uso de dataclasses, que favorecen la mantenibilidad. Ambos modelos cumplieron con los requisitos, generando documentación completa y comprendiendo con precisión el problema planteado. En las pruebas con NumPy, las diferencias de tiempo fueron mínimas. Se concluye que GPT-5.0 ofrece mayor eficiencia y explicaciones más detalladas, y Claude Sonnet 4 resalta en diseño de código moderno. El estudio evidencia el potencial de la IA generativa como asistente técnico clave en tareas algorítmicas complejas.

Introducción

La inteligencia artificial (IA) generativa ha emergido como una herramienta transformadora en el campo del desarrollo de software. Modelos de lenguaje avanzados, como los que impulsan a ChatGPT y Claude, ahora son capaces de comprender requisitos complejos y generar código fuente funcional en diversos lenguajes de programación. Esta capacidad promete acelerar los ciclos de desarrollo, optimizar el rendimiento del código y asistir a los programadores en tareas algorítmicas complejas.

El presente informe tiene como objetivo evaluar y comparar la eficacia de distintas herramientas de IA generativa en una tarea específica y de alta relevancia en la computación científica: la multiplicación de matrices de manera concurrente. Se analizará no solo la funcionalidad del código generado, sino también su rendimiento, calidad, y la interacción necesaria para refinar los resultados. A través de este estudio, se busca obtener una visión clara sobre las capacidades, ventajas y limitaciones de las IAs actuales como asistentes en el desarrollo de software de alto rendimiento.

Desarrollo

Para llevar a cabo este estudio comparativo, se definió un proceso metodológico riguroso, desde la selección de las herramientas hasta la definición de las métricas de evaluación.

Entorno de ejecución

Todas las pruebas se realizaron en una PC de escritorio configurada con la siguiente arquitectura:

- Procesador Intel(R) Core(TM) i7-7700K CPU @ 4.20GHz 4.20 GHz
- RAM instalada 16,0 GB (3200Mhz)
- Tarjeta gráfica NVIDIA GeForce RTX 3080 (10 GB)
- Tipo de sistema Sistema operativo de 64 bits, procesador basado en x64
- Edición Windows 10 Pro
- Versión 22H2
- Compilación del sistema operativo 19045.6332

La plataforma utilizada fue Python ejecutado en un entorno local, con la biblioteca estándar habilitada y sin el uso de frameworks externos de alto nivel. Las pruebas de rendimiento se llevaron a cabo en condiciones controladas, asegurando la estabilidad del sistema y la repetibilidad de los resultados obtenidos.

Herramientas Seleccionadas

Se seleccionaron dos de los modelos de lenguaje más avanzados disponibles en el momento del estudio, bajo los nombres con los que se identificaron los resultados:

- GPT-5.0: Representa la familia de modelos desarrollados por OpenAI. Es conocido por su capacidad para generar código complejo y seguir instrucciones detalladas.
- Claude Sonnet 4: Representa la familia de modelos de Anthropic. Se destaca por su enfoque en la seguridad y su habilidad para mantener conversaciones coherentes y detalladas.
- GitHub Copilot: Aunque no se generó un conjunto de código separado bajo este nombre, se considera una herramienta fundamental en el ecosistema de desarrollo. GitHub Copilot, impulsado por modelos de OpenAI, se integra directamente en el editor de código y ofrece sugerencias en tiempo real. Su API

y capacidades son relevantes para el contexto general del desarrollo asistido por IA.

Características de las Herramientas Seleccionadas

- **ChatGPT y GPT-5.0:** ChatGPT es una interfaz de conversación que utiliza los modelos de la serie GPT (Generative Pre-trained Transformer). Estos modelos son entrenados en vastos conjuntos de datos de texto y código, permitiéndoles comprender el contexto, la lógica de programación y generar soluciones algorítmicas. "GPT-5.0" en este informe se refiere a una versión avanzada hipotética o actual que demuestra una alta capacidad de generación de código.
- **Claude y Sonnet 4:** Claude es la familia de modelos de IA de Anthropic. La versión "Sonnet 4" representa un modelo potente y equilibrado, diseñado para manejar tareas complejas de razonamiento y generación de código a gran escala, manteniendo al mismo tiempo un enfoque en la fiabilidad y la seguridad.
- **GitHub Copilot:** Es un asistente de programación que se integra en entornos de desarrollo como VS Code. Utiliza modelos de OpenAI para ofrecer autocompletado de código inteligente, sugerir funciones enteras o bloques de código, y ayudar en la depuración. Su principal diferencia es la integración directa en el flujo de trabajo del desarrollador.

Prompt Técnico Utilizado

Para asegurar que las IAs recibieran los mismos requisitos, se elaboró un prompt técnico estandarizado y detallado:

- **LENGUAJE DE PROGRAMACIÓN:** python
- **PLATAFORMA:** x86
- **FUNCIONALIDAD A IMPLEMENTAR:** Desarrollar un algoritmo que multiplique matrices de manera concurrente.
- **REQUISITOS ESPECÍFICOS:** Se debe implementar, además del algoritmo de multiplicación de matrices, una manera de censar cuánto se tarda en hacer dichas multiplicaciones (benchmarking), un contador de hilos/procesos concurrentes y la cantidad de cálculos realizados. El código debe poder trabajar en matrices de igual dimensión, pero también de diferentes, sin limitar el tamaño de las mismas.
- **RESTRICCIONES:** Usar librerías estándar de Python o código propio. El algoritmo no puede ser realizado con librerías

externas de alto nivel como NumPy para la operación de multiplicación principal.

- **EXTRA:** Puedes hacernos cualquier pregunta necesaria para que esto salga de la mejor manera posible.

Tipo de Prompt

El prompt utilizado es de tipo "Zero-Shot con Requisitos Detallados". No se proporcionó a la IA ningún ejemplo previo de código (lo que lo haría "Few-Shot"). En su lugar, se le entregó un conjunto de instrucciones claras, estructuradas y específicas que definen el problema, los objetivos, las funcionalidades esperadas y las restricciones. La sección "EXTRA" fomenta una interacción de refinamiento, permitiendo a la IA solicitar aclaraciones, una característica clave para resolver problemas complejos.

Actividades Iniciales Realizadas en Python

Una vez obtenido el código de las IAs, se procedió a la ejecución y análisis. Las actividades iniciales incluyeron:

1. **Ejecución de Pruebas:** Se corrieron los scripts `matrix_concurrent.py` y `matrix_concurrent_numpy.py` con diferentes dimensiones de matrices y configuraciones de concurrencia.
2. **Generación de Métricas:** Se utilizaron los propios scripts para generar benchmarks, como los archivos `scaling1.json` y `scaling2.json`, que miden el tiempo de ejecución bajo diferentes cargas y modos de operación.
3. **Análisis de READMEs:** Se revisaron los archivos `README.md` generados por las IAs para comprender la arquitectura, el uso y las limitaciones del código propuesto.

Resultados y Objetivos

El objetivo principal era obtener dos implementaciones funcionales y comparables: una en Python puro y otra utilizando NumPy para operaciones optimizadas. Ambas IAs lograron este objetivo con éxito.

Resultados Obtenidos en GPT-5.0

La interacción con el modelo GPT-5.0 fue directa y eficiente. Tras recibir el prompt inicial, el modelo generó una solución robusta que cumplía con todos los requisitos.

- **matrix_concurrent.py:** Implementó los modos naive (división por filas) y blocked (división por tiles/bloques), demostrando una comprensión de las optimizaciones de localidad de caché. Permitió la concurrencia mediante `ThreadPoolExecutor` y

ProcessPoolExecutor.

- `matrix_concurrent_numpy.py`: Creó una versión optimizada con NumPy, incluyendo un modo `direct` ($A @ B$), un modo `blocked` educativo y un modo `hybrid-process`. Demostró conocimiento avanzado al advertir sobre la sobre-suscripción de hilos y la necesidad de configurar variables de entorno como `OMP_NUM_THREADS=1`.
- Documentación: El `readme_concurrent.md` fue muy detallado, explicando los objetivos, modos, limitaciones de memoria y rendimiento, y proporcionando ejemplos de uso avanzados.

El ida y vuelta fue mínimo; el modelo interpretó correctamente la necesidad de separar la lógica de Python puro de la de NumPy y de incluir un sistema de benchmarking completo.

Resultados Obtenidos en Claude Sonnet 4

Claude Sonnet 4 también proporcionó una solución completa y bien estructurada, aunque con un enfoque ligeramente diferente en la organización del código.

- `matrix_concurrent.py`: Al igual que GPT, implementó los modos `naive` y `blocked`. La estructura del código fue clara, utilizando `dataclasses` para organizar los datos de resultados, lo que mejora la legibilidad.
- `matrix_concurrent_numpy.py`: La versión de NumPy también fue muy completa. Incluyó un modo `hybrid-process` que utiliza `ProcessPoolExecutor` para dividir la tarea, una técnica válida para ciertos escenarios de hardware.
- Documentación: El `README_MATRIX.md` fue claro y conciso, cubriendo las características principales, el uso básico, los parámetros y ejemplos de salida JSON.

La interacción con Claude fue igualmente fluida, requiriendo pocas aclaraciones para llegar a un resultado final de alta calidad.

Métricas de Evaluación

Para comparar los resultados de manera objetiva, se definieron métricas de eficiencia (rendimiento) y de calidad de código (mantenibilidad).

Métricas de Eficiencia y Rendimiento

Se analizaron los archivos JSON de benchmark para una multiplicación de matrices de 1000×1000 .

- Python Puro (Modo `naive`, 8 workers):
 - GPT-5.0 (`scaling1.json`): Tiempo promedio de 32.10 segundos.
 - Claude Sonnet 4 (`scaling1.json`):

Tiempo promedio de 42.56 segundos.

- Análisis: En la implementación de Python puro, el código generado por GPT-5.0 fue aproximadamente un 24% más rápido.
- Python Puro (Modo `blocked`, 8 workers, tiles 96×96):
 - GPT-5.0 (`scaling2.json`): Tiempo promedio de 41.05 segundos.
 - Claude Sonnet 4 (`scaling2.json`): Tiempo promedio de 48.26 segundos.
 - Análisis: El código de GPT-5.0 fue un 15% más rápido en el modo `blocked`. Curiosamente, para ambas IAs, el modo `blocked` fue más lento que el `naive`, lo que podría indicar que la sobrecarga de la gestión de tiles superó los beneficios de localidad de caché para estas dimensiones y estrategia de implementación.
- NumPy (Modo `direct`):
 - GPT-5.0 (`scaling1_numpy.json`): Tiempo de multiplicación de 2.01 segundos.
 - Claude Sonnet 4 (`scaling1_numpy.json`): Tiempo de multiplicación de 1.89 segundos.
 - Análisis: En la multiplicación directa con NumPy, que depende de librerías BLAS/LAPACK subyacentes altamente optimizadas, los resultados son muy similares. Claude Sonnet 4 fue marginalmente más rápido, pero la diferencia es mínima y puede deberse a fluctuaciones del sistema.

Métricas de Calidad y Precisión del Código

- Estructura y Claridad:
 - GPT-5.0: Utilizó un enfoque más funcional, con funciones bien definidas y separadas. La lógica es directa y fácil de seguir.
 - Claude Sonnet 4: Hizo un excelente uso de `dataclasses` (`TimingData`, `MatrixResult`), lo que mejora la estructura de los datos y la autodocumentación del código. Este enfoque es ligeramente superior en términos de principios de diseño de software moderno.
- Documentación (`Docstrings` y Comentarios):
 - Ambos modelos generaron

excelente documentación. Los docstrings son claros, explican los parámetros, los retornos y las excepciones.

- El código de GPT-5.0 incluye comentarios adicionales dentro de las funciones que explican decisiones de implementación específicas (ej., por qué se transpone la matriz B).
- **Mantenibilidad:**
 - El uso de dataclasses por parte de Claude Sonnet 4 hace que el código sea potencialmente más fácil de mantener y extender, ya que las estructuras de datos están explícitamente definidas.
 - El código de GPT-5.0 es muy modular, lo que también facilita la mantenibilidad. La elección entre uno y otro es más una preferencia de estilo.
- **Funcionalidades Adicionales:**
 - Ambos modelos cumplieron con todos los requisitos del prompt.
 - GPT-5.0 añadió una advertencia explícita en su README sobre el GIL y la recomendación de usar process para tareas intensivas en CPU, demostrando una comprensión profunda de las particularidades de Python.

Conclusiones

Este estudio demuestra que las herramientas de IA generativa de última generación, como las representadas por GPT-5.0 y Claude Sonnet 4, son capaces de producir código de alta calidad para tareas algorítmicas complejas como la multiplicación de matrices concurrentes.

1. **Rendimiento:** En términos de rendimiento puro del código generado en Python, la implementación de GPT-5.0 fue consistentemente más rápida. Sin embargo, cuando se utiliza una librería optimizada como NumPy, las diferencias de rendimiento se vuelven insignificantes, ya que el trabajo pesado es delegado a la misma capa subyacente.
2. **Calidad del Código:** Ambas IAs produjeron código bien estructurado, documentado y mantenible. Claude Sonnet 4 destacó por su uso de dataclasses, que es una práctica moderna y recomendable. GPT-5.0 brilló por su documentación detallada y sus explicaciones técnicas profundas en los comentarios y READMEs.
3. **Comprensión del Prompt:** Ambos modelos interpretaron el prompt con una precisión excepcional, cumpliendo todos los requisitos y restricciones sin necesidad de

un largo ciclo de refinamiento.

En resumen, no hay un "ganador" absoluto. La elección de una herramienta sobre otra podría depender de la prioridad del proyecto: si se busca el máximo rendimiento en implementaciones personalizadas, GPT-5.0 podría tener una ligera ventaja. Si se prioriza un diseño de código moderno y estructuras de datos explícitas, Claude Sonnet 4 ofrece una solución ejemplar. El futuro del desarrollo de software se perfila como una colaboración cada vez más estrecha entre humanos y máquinas, donde la habilidad residirá no solo en escribir código, sino en formular los problemas y requisitos de manera precisa para que estas potentes herramientas puedan asistirnos en la creación de soluciones eficientes y robustas.

Referencias

- *P. Python Software Foundation, "Documentación oficial de Python sobre concurrent.futures", Python Documentation, [en línea], Disponible en: <https://docs.python.org/3/library/concurrent.futures.html>, Consultado: octubre 2025.*
- *T. Harris, C. R. Harris et al., "Documentación oficial de la librería NumPy", NumPy Documentation, [en línea], Disponible en: <https://numpy.org/doc/stable/>, Consultado: octubre 2025.*
- *UNLAM-PROG-C, Archivos de código y benchmarks generados, disponibles en el repositorio UNLAM-PROG-C/2025-PROGC-Q2-M4, [en línea], GitHub, Consultado: octubre 2025.*